

파이썬 데이터 분석

데이터 유형에 따른 분석 기법 선택

1. 정형 데이터 (Structured Data) : 행과 열로 정리된 데이터 (예: 엑셀, DB 테이블, CSV, TSV)

- 분석 도구/기법:
 - SQL 쿼리 (집계, 조인, 필터링)
 - 파이썬 **pandas** (데이터프레임 분석)
- 활용 예시: 매출 분석(날짜, 상품명, 가격, 수량), 고객 통계(이름, 나이, 연락처)

2. 반정형 데이터 (Semi-Structured Data) : 일정한 구조 있지만 자유로운 형태 (예: JSON, XML, 로그 파일)

- 분석 도구/기법:
 - SQL의 JSON 처리 기능
 - 파이썬 **json**, **pandas**로 변환 후 분석
- 활용 예시: 웹 로그(`{"user": "kim", "action": "click"}`), API 응답 데이터
(`<product><name>TV</name><price>500</price></product>`)

3. 비정형 데이터 (Unstructured Data) : 정해진 틀이 없음 (예: 텍스트, 이미지, 동영상)

- 분석 도구/기법:
 - 파이썬 **NLP** 라이브러리 (텍스트 마이닝, 형태소분석, 임베딩)
 - 이미지·영상: AI/머신러닝 활용
- 활용 예시: 리뷰 감성 분석("이 제품 진짜 좋아요"), 사진 분류(상품 사진, 홍보 영상)

파이썬 데이터 분석에 유용한 모듈 NumPy, Pandas

1. NumPy 넘파이

Numerical Python의 약자로 과학 및 공학 분야에서 사용되는 오픈 소스 파이썬 라이브러리

- 대량의 데이터를 다루는 배열을 만들어주는 라이브러리
- 파이썬 배열(리스트)와 다르게 수학적 행렬의 사칙연산을 수행해 줌.

1.1 넘파이 설치

```
$ pip install numpy
```

아나콘다 주피터노트북 or 코랩(Colab)환경에는 이미 넘파이 모듈이 설치되어 있음.

1.2 넘파이 import

```
import numpy
```

사용 편의를 위해 보통 이름을 짧게 지정하여 사용함

```
import numpy as np
```

1.3 넘파이 배열 만들기

```
np.array([10,20,30])
```

파이썬 리스트(배열) .vs. 넘파이 배열

1. 파이썬 리스트(배열)

- 리스트(배열)의 연산은 산술연산이 아님.

```
a = [10,20,30]
```

```
b = [4,5,6]
```

```
c = a + b
```

결과 : [10, 20, 30, 4, 5, 6] #요소의 덧셈이 아닌 리스트의 결합이 됨.

2. 넘파이 배열

- 배열 요소들의 산술연산이 가능함. 수학의 행렬 연산과 유사함.

```
import numpy as np
```

```
a= np.array([10, 20 ,30])
```

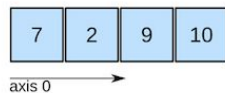
```
b= np.array([4, 5, 6])
```

```
c= a+b
```

결과 : [14, 25, 36] #요소끼리 덧셈연산 수행

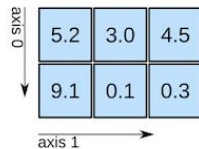
넘파이 배열의 차원

1D array



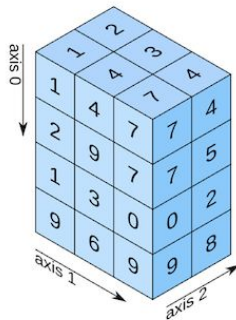
shape: (4,)

2D array



shape: (2, 3)

3D array

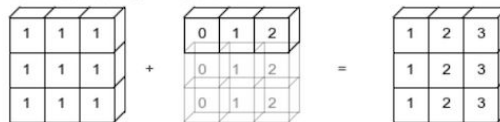


shape: (4, 3, 2)

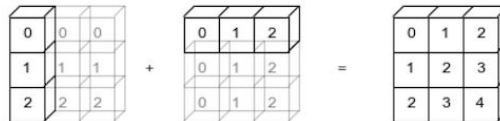
`np.arange(3)+5`



`np.ones((3, 3)) + np.arange(3)`



`np.arange(3).reshape((3, 1)) + np.arange(3)`



```
arr = np.array(12)
```

0차원 배열 (스칼라, **Scalar**)

```
arr = np.array([1, 3, 5])
```

1차원 배열 (벡터, **Vector**)

```
arr = np.array([[2, 4, 6], [1, 3, 5]])
```

2행 3열, 2차원 배열 (행렬, **Matrix**)

```
arr = np.array([[[111, 112, 113], [121, 122, 123]],
```

```
[[211, 212, 213], [221, 222, 223]])
```

3차원 배열 (텐서, **Tensor**)

넘파이 주요 메소드와 속성

배열 생성

`print(np.zeros(5))` # 출력 : [0. 0. 0. 0. 0.] → 0 으로 채워진 1차원 배열 생성

`print(np.ones(5))` # 출력 : [1. 1. 1. 1. 1.] → 1 로 채워진 1차원 배열 생성

`print(np.full(5, 7))` # 출력 : [7 7 7 7 7] → 7 로 5 개 채워진 1차원 배열 생성

`print(np.arange(5))` # 출력 : [0 1 2 3 4] → 0 부터 시작하는 5 개의 숫자 생성 (0, 1, 2, 3, 4)

`print(np.arange(2,20,3))` # 출력 : [2 5 8 11 14 17] → 일정한 간격의 배열 생성, (시작, 정지, 단계)

`print(np.random.random(3))` # 출력 : 0 이상 1 미만의 난수 3개를 랜덤하게 배열 생성

`print(np.linspace(0,100, num=3))` # 출력 : [0. 50. 100.] → 선형 간격으로 배치된 값으로 배열 생성

배열 정보

`.shape` 배열의 구조 (행, 열) 반환

`.ndim` 배열의 차원 수 반환

`.size` 배열의 전체 요소 수

`.dtype` 배열의 자료형 반환

파이썬 데이터 분석에 유용한 모듈 Pandas

2. Pandas 판다스

데이터 작업에 사용되는 오픈 소스 파이썬 라이브러리

- 데이터 분석, 정리, 탐색, 조작을 위한 기능이 포함
- 2차원 데이터프레임 형태로 구성된 데이터를 분석하기 좋은 형태로 전처리 하는 기능을 제공

2.1 판다스 설치

```
$ pip install pandas
```

아나콘다 주피터노트북 or 코랩(Colab)환경에는 이미 판다스 모듈이 설치되어 있음.

2.2 판다스 import

```
import pandas
```

사용 편의를 위해 보통 이름을 짧게 지정하여 사용함

```
import pandas as pd
```

Pandas 데이터 구조

1. Series 시리즈

순차데이터를 저장하는 1차원 배열로 구성. 단일타입(숫자 또는 문자)의 데이터로 구성

예: [10, 20, 30]

```
import pandas as pd
city = ['서울', '부산', '울산']
df = pd.Series(city)      # Series 메서드를 사용하여 시리즈를 생성한다.
```

1. Dataframe 데이터프레임

행렬로 구성된 2차원 배열로 구성. 각 열에 서로 다른 타입의 데이터로 구성 가능.

예: [['sam', 20], ['robin', 30]]

```
import pandas as pd
city = {
    '도시': ['서울', '부산', '울산'],
    '인구': [940, 330, 110]
}
df = pd.DataFrame(city)
```

[Pandas 실습] 데이터프레임 만들기

```
import pandas as pd

data = [['김민재', 27, 75, 5428000],
        ['이강인', 22, 57, 3428000],
        ['박찬호', 50, 91, 8428000],
        ['차범근', 70, 80, 4428000],
        ['추신수', 43, 100, 4528000],
        ['손흥민', 31, 72, 7028000],
        ['황희찬', 28, 69, 2528000]]
```

```
df = pd.DataFrame(data, columns=['성명', '나이', '몸무게', '급여'])

df
```

‘성명’ 컬럼의 데이터

```
df['성명']    # Series 형태로 결과 출력

df.성명      # 대괄호[ ]를 대신해서 점(.)을 사용하여 볼 수도 있음.

df[['성명']] # 이중 대괄호[[ ]]로 입력하면 데이터프레임 형태로
출력
```

	성명	나이	몸무게	급여
0	김민재	27	75	5428000
1	이강인	22	57	3428000
2	박찬호	50	91	8428000
3	차범근	70	80	4428000
4	추신수	43	100	4528000
5	손흥민	31	72	7028000
6	황희찬	28	69	2528000

여러 컬럼의 데이터

```
df[['성명'], ['급여']] # 성명, 급여 컬럼 한번에 출력

- 대괄호 내에 반드시 리스트 형태(대괄호 포함)로 입력
- df['성명', '급여']는 리스트 형태(대괄호)가 아니기 때문에 에러 발생
```

[Pandas 실습] 특정 행(row) 데이터 .loc location

.loc는 location의 약자로 하나 이상의 지정된 행과 열을 표시

.loc는 인덱스 뿐만 아니라 슬라이싱이 가능

.loc 사용 시 **[]**는 시리즈 형태로 보여주며, **[[]]**는 데이터프레임 형태로 표시

.loc[행, 열]에서 행은 숫자로 인덱싱하며, 컬럼은 컬럼명으로 인덱싱

```
.loc[[1], ['성명']]
```

`df.loc[0]` # 인덱스 0번 행의 데이터를 시리즈 형태로 표시

`df.loc[[0]]` # 데이터프레임 형태로 표시

`df.loc[[0,1,3]]` # 0, 1, 3번 행을 한꺼번에 데이터프레임 형태로 표시
지정한 대로 여러 행을 한꺼번에 보려면 대괄호를 포함

`df.loc[0:3]` # 인덱스 0번 행에서 인덱스 3번 행까지의 데이터 4개를 표시

`df.iloc[0:3]` # 인덱스 0번 행에서 인덱스 3번 행 앞까지의 데이터 3개를 표시

	성명	나이	몸무게	급여
0	김민재	27	75	5428000
1	이강인	22	57	3428000
2	박찬호	50	91	8428000
3	차범근	70	80	4428000
4	추신수	43	100	4528000
5	손흥민	31	72	7028000
6	황희찬	28	69	2528000

특정 행의 특정 컬럼 데이터 표시

`df.loc[0, '성명']` # 0번행의 성명 값 표시 - 시리즈 형태 '김민재'

`df.loc[[0], ['성명']]` # 0번행의 성명 값 표시 - 데이터프레임 형태
'김민재'

`df.loc[0,1, '성명']` # 0번, 1번행의 성명 값 표시 - 시리즈 형태

`df.loc[[0,1], ['성명']]` # 0번, 1번행의 성명 값 표시 - 데이터프레임

형태 `df.loc[[0,2], ['성명','나이']]` #인덱스 0과 2번 행에서 '성명'과 '나이' 컬럼 표시

`df.loc[0:3, ['성명','급여']]` # 인덱스 0~3번 행에서 '성명' '급여' 컬럼 표시

[Pandas 실습] 조건에 맞는 데이터 표시

.loc[조건, 컬럼] 메서드를 통해 조건에 맞는 특정 행과 열을 표시

1) 결과를 True/False로 표시

급여가 500만 이상인 경우에 True/False로 표시

```
df['급여'] >= 5000000
```

AND(&) 조건 : 급여가 500만 이상이고, 나이가 40세 미만

```
(df['급여'] >= 5000000) & (df['나이'] < 40)
```

OR(|) 조건 : 급여가 500만 이상이거나, 몸무게가 90 이상

```
df[(df['급여'] >= 5000000) | (df['몸무게'] > 90)]
```

NOT(~)급여가 500만 이상이 아닌 경우(500만 미만인 경우)

```
~(df['급여'] >= 5000000)
```

2) 조건에 맞는 행, 열만 표시

'급여' 컬럼의 데이터에서 5백만원 이상인 행을 찾아 표시

```
df.loc[df['급여'] >= 5000000]
```

'나이' 컬럼의 데이터에서 30 이상이고 35 이하인 행을 표시

```
df.loc[(df['나이'] >= 30) & (df['나이'] <= 35)]
```

'급여' 컬럼의 데이터에서 5백만원 이상인 행에 대해 '성명' 컬럼만을 표시

```
df.loc[df['급여'] > 5000000, ['성명']]
```

```
df.loc[df['급여'] > 5000000]['성명']
```

'몸무게' 컬럼의 데이터에서 80 이상인 행을 추출한 다음, 해당 행에 대해서만 '체질량' 컬럼에 '비만'을 입력

```
df.loc[df['몸무게'] > 80, '체질량'] = '비만'
```

	성명	나이	몸무게	급여
0	김민재	27	75	5428000
1	이강인	22	57	3428000
2	박찬호	50	91	8428000
3	차범근	70	80	4428000
4	추신수	43	100	4528000
5	손흥민	31	72	7028000
6	황희찬	28	69	2528000

[Pandas 실습] 문자열 다루기 .str

판다스는 문자열로 구성된 데이터에서 **특정 문자열을 분할, 변경, 대체, 추출, 제거 등의 작업**이 가능

str.split() : 문자열을 특정 구분자를 기준으로 분할한다.

str.upper() : 문자열을 대문자로 변환한다.

str.replace() : 문자열 내의 특정 부분을 다른 문자열로 대체한다.

str.contains() : 문자열에 포함된 특정 문자열이 포함되어 있는지 여부를 **True/False**로 반환한다.

str.strip() : 문자열 양 끝에 포함된 공백을 제거한다.

[Pandas 실습] 문자열 다루기 .str

```
import pandas as pd

data = [['김예찬', 'm', '부장', '010-1234-5678', '서울시 강남구  
삼성동'],

        ['이하은', 'f', '인턴', '010-8765-4321', '부산시 해운대구  
우동'],

        ['박지현', 'f', '과장', '010-1357-2468', '대구시 수성구  
범어동'],

        ['최준호', 'm', '임시', '010-1928-3746', '인천시 연수구  
송도동'],

        ['김예서', 'f', '과장', '010-2846-5937', '천안시 서북구  
두정동']]

df = pd.DataFrame(data)

df.columns = ['성명', '성별', '직급', '전화번호', '주소']
```

'성별' 컬럼에 포함된 소문자 'm', 'f' 를 대문자 'M', 'F' 로

```
df['성별'] = df['성별'].str.upper()
```

'성별' 컬럼에 포함된 'M', 'F' 를 'Male', 'Female' 로

```
df['성별'] = df['성별'].str.replace('M', 'Male')
```

```
df['성별'] = df['성별'].str.replace('F', 'Female')
```

'직급' 컬럼에 포함된 '인턴' 과 '임시' 를 '사원'으로

```
df['직급'] = df['직급'].str.replace('인턴|임시','사원', regex=True)
```

'성별' 컬럼에서 '여성' 이라는 문자열이 포함되어 있는지 여부 True,
False로 반환

```
df['성별'].str.contains('여성')
```

df[] 는 특정 코드의 결과로 얻은 True 인 행만 필터링하여 표시

```
df[df['성별'].str.contains('여성')]
```

```
df['도시'] = df['주소'].str[:4]      # '주소' 열에서 앞 4글자만 추출
```

```
df['주소'] = df['주소'].str.strip()  # '주소' 컬럼에 포함된 불필요한 공백
```

[Pandas 실습] CSV 파일 다루기

판다스에서 호출할 수 있는 파일 형식

파일 형식	Reader	Writer
CSV	read_csv	to_csv
EXCEL	read_excel	to_excel
HTML	read_html	to_html
JSON	read_json	to_json
SQL	read_sql	to_sql

```
import pandas as pd
```

```
df = pd.read_csv('/pandas.csv', encoding='CP949')
```

```
df
```

	A	B	C	D	E
1	번호	성명	나이	몸무게	급여
2	100	김민재	27	75	5428000
3	101	이강인	22	57	
4	102	박찬호	50	91	8428000
5	103	차범근	70		4428000
6	104	추신수	43	100	4528000
7	105	손흥민	31	72	7028000
8	106	황희찬	28	69	2528000

[Pandas 실습] CSV 파일 데이터 프레임 정보 확인하기

- csv 파일의 컬럼명(필드명)을 확인

```
df.columns
```

- csv 파일의 행과 컬럼의 크기를 확인

```
df.shape
```

- csv 파일의 각 컬럼에 대한 데이터 형식을 확인

```
df.dtypes
```

- csv 파일의 '몸무게' 컬럼에 대한 데이터 형식을 정수(int64) 형식으로

```
df['몸무게'] = df['몸무게'].astype('Int64')
```

```
df.info() # 각 컬럼의 데이터 형식을 확인
```

	A	B	C	D	E
1	번호	성명	나이	몸무게	급여
2	100	김민재	27	75	5428000
3	101	이강인	22	57	
4	102	박찬호	50	91	8428000
5	103	차범근	70		4428000
6	104	추신수	43	100	4528000
7	105	손흥민	31	72	7028000
8	106	황희찬	28	69	2528000

[Pandas 실습] CSV 파일 데이터 내용 확인하기

	A	B	C	D	E
1	번호	성명	나이	몸무게	급여
2	100	김민재	27	75	5428000
3	101	이강인	22	57	
4	102	박찬호	50	91	8428000
5	103	차범근	70		4428000
6	104	추신수	43	100	4528000
7	105	손흥민	31	72	7028000
8	106	황희찬	28	69	2528000

- 상위 5개 행의 데이터
표시

```
df.head()
```

- 상위 3개 행의 데이터
표시

```
df.head(3)
```

- 하위 5개 행의 데이터
표시

```
df.tail()
```

- 하위 3개 행의 데이터

```
df.loc[0:3, ['성명', '급여']] # 인덱스 0~3번 행에 대한 '성명'과 '급여' 컬럼을 표시
```

```
df[df['급여']>=5000000] # '급여' 컬럼의 데이터에서 5백만원 이상인 행을 표시
```

```
# '급여' 컬럼의 데이터에서 5백만원 이상인 행에 대한 '번호', '성명', '급여' 컬럼을 표시  
df.loc[df['급여']>=5000000, ['번호', '성명', '급여']]
```

```
# '급여' 컬럼의 데이터에서 5백만원 이상인 행에 대한 '번호'에서 부터 '급여'까지의 컬럼을 표시  
df.loc[df['급여']>=5000000, '번호':'급여']
```

```
df['성명']=='손흥민' # '성명' 컬럼에서 '손흥민' 인 데이터에 대해 True, False 값을  
표시
```

```
df[df['성명']=='손흥민'] # True 에 해당하는 행의 데이터를 표시
```

```
df[(df['성명']=='손흥민') | (df['성명']=='이강인')] # '성명' 컬럼에서 '손흥민' 또는 '이강인' 데이터를  
표시
```

```
df['성명'].isin(['박찬호', '추신수']) # .isin() 메서드는 특정 값과 일치하는지 여부를 True, False  
로 표시
```

```
df[df['성명'].isin(['박찬호', '추신수'])] # '박찬호'와 '추신수' 데이터만을 표시
```

```
# '급여' 컬럼의 데이터에서 5백만원 이상인 행만 추출하여 '고액연봉자' 변수에 저장  
고액연봉자 = df[df['급여']>=5000000]  
고액연봉자
```

[Pandas 실습] CSV 파일 데이터 정렬 및 결측치

	A	B	C	D	E
1	번호	성명	나이	몸무게	급여
2	100	김민재	27	75	5428000
3	101	이강인	22	57	
4	102	박찬호	50	91	8428000
5	103	차범근	70		4428000
6	104	추신수	43	100	4528000
7	105	손흥민	31	72	7028000
8	106	황희찬	28	69	2528000

'나이' 컬럼을 기준으로 데이터를 정렬한다.

```
df.sort_values(by='나이')
```

'나이' 컬럼을 기준으로 데이터를 역순으로 정렬한다.

```
df.sort_values('나이', ascending=False)
```

'나이' 컬럼을 기준으로 정렬한 다음, '몸무게' 컬럼을 기준으로 정렬

```
df.sort_values(['나이', '몸무게'])
```

결측치란

- 결측치 (Missing values)란 데이터셋에서 일부 값이 누락된 것
- 결측치는 데이터 분석 왜곡, 통계 분석 문제, 기계학습 모델 문제 등을 야기할 수 있기 때문에 반드시 데이터 분석 전에 전처리 작업을 해 주어야 함.
- 데이터 전처리 작업은 먼저 결측치가 있는 행 또는 열을 찾는 것에서 시작
- 판다스는 결측치가 있는 행 또는 열을 찾아 True, False 로 표시해 주는 메서드를 제공

결측치 처리를 위한 메서드

.isna() : 결측치가 존재하면 True, 존재하지 않으면 False 반환 .isnull() 메서드와 동일

.notna() : 결측치가 존재하지 않으면 True, 결측치가 존재하면 False 반환
.notnull()

[Pandas 실습] CSV 파일 데이터 결측치 확인 및 제거하기

	A	B	C	D	E
1	번호	성명	나이	몸무게	급여
2	100	김민재	27	75	5428000
3	101	이강인	22	57	
4	102	박찬호	50	91	8428000
5	103	차범근	70		4428000
6	104	추신수	43	100	4528000
7	105	손흥민	31	72	7028000
8	106	황희찬	28	69	2528000

결측치 제거하기

columns 매개변수를 사용하여 결측치가 존재하는
'몸무게' 컬럼과 '급여' 컬럼을 한 번에 삭제
`df.drop(columns = ['몸무게', '급여'])`

결측치가 존재하는 인덱스 1번과 3번 행을 한 번에 삭제
`df.drop(index = [1, 3])`

결측치가 있는 행 전체를 한 번에 삭제
`df.dropna()`

결측치 확인하기

결측치가 존재하는 곳에 True를 표시
`df.isna()`

.isna() 메서드와 같이 결측치가 존재하는 곳에 True를 표시
`df.isnull()`

전체 데이터에서 각 컬럼마다 존재하는 결측치를 더한 결과를 표시
`df.isna().sum()`

'몸무게' 컬럼에서 결측치가 존재하는 행을 필터링
`df[df['몸무게'].isna()]`

'몸무게' 컬럼에서 결측치가 있는 행을 True, False로 찾은 다음, True에 해당하는 '성명' 컬럼을
필터링하여 표시
`df[df['몸무게'].isna() | df['급여'].isnull()]`

결측치가 존재하지 않는 데이터를 표시
`df[df['몸무게'].notna()]`

'몸무게' 컬럼과 '급여' 컬럼에서 결측치가 존재하는 행을 제외한 나머지 행을 필터링하여 표시
`df[df['몸무게'].notna() & df['급여'].notnull()]`

[Pandas 실습] CSV 파일 데이터 그룹화하기

	A	B	C	D	E	F	G
1	번호	판매원	날짜	제품	판매개수	판매액	지역
2	1	서태웅	2021-04-01	립글로스	45	360	남부
3	2	이달재	2021-03-10	파운데이션	50	250	중서부
4	3	강백호	2022-02-25	립스틱	9	90	중서부
5	4	이달재	2023-05-22	립글로스	55	440	서부
6	5	정병욱	2021-06-17	립글로스	43	344	중서부
7	6	송태섭	2022-11-27	아이라이너	58	406	중서부
8	7	정대만	2021-03-21	아이라이너	8	56	중서부
9	8	송태섭	2023-12-17	립글로스	72	576	중서부
10	9	강백호	2023-07-05	아이라이너	75	525	남부
11	10	서태웅	2023-08-07	립글로스	24	192	동부

```
import pandas as pd
```

```
df = pd.read_csv('pandas_groupby.csv', encoding='CP949')
```

```
df
```

```
df.head(2) # 상위 2개 행 표시
```

그룹화 하기 전에 먼저 각 컬럼에 저장된 고유한 값(unique values)을 확인

```
df['지역'].unique() # '지역' 컬럼에는 4개의 지역이 저장되어 있음
```

'지역' 컬럼을 기준으로 그룹화

```
df.groupby(by='지역') # 'by=' 매개변수는 생략 가능
```

일반적으로 .groupby() 메서드는 그룹별 통계 데이터를 보기 위해 사용
mean(평균), sum(합계), count(개수) 등

'지역' 별로 그룹화한 다음, 각 지역에 대한 개수를 표시

```
df.groupby('지역').count()
```

mean() 함수는 여러 발생, 평균은 숫자형 컬럼에서 사용

'지역' 별로 '판매개수'에 대한 평균을 표시

```
df.groupby('지역')['판매개수'].mean()
```

데이터프레임 형태로 '지역' 별 판매개수의 평균을 표시

'지역' 별로 판매된 '제품'에 대한 '판매개수'의 합계를 알고자할 경우,
먼저 '지역' 컬럼을 기준으로 1차 그룹화한 다음 '제품' 컬럼을 기준으로
2차 그룹화

```
df.groupby(['지역', '제품'])['판매개수'].sum()
```

'판매원' 별로 각 '제품'을 얼마나 많이 판매했는지 '판매개수'와
'판매액'의 합계

```
df.groupby(['판매원', '제품'])['판매개수', '판매액'].sum()
```

[Pandas 실습] CSV 파일 데이터 통계 구하기

	A	B	C	D	E	F
1	번호	성명	나이	몸무게	급여	종목
2	100	김민재	27	75	5428000	축구
3	101	이강인	22	57	3428000	축구
4	102	박찬호	50	91	8428000	야구
5	103	차범근	70	80	4428000	축구
6	104	추신수	43	100	4528000	야구
7	105	손흥민	31	72	7028000	축구
8	106	황희찬	28	69	2528000	축구

```
import pandas as pd
```

```
df = pd.read_csv('pandas_stat.csv', encoding='CP949')
```

```
df
```

```
df.head(2) # 상위 2개 행 표시
```

```
print('최대값 = ', df['급여'].max()) # 최대값

print('최소값 = ', df['급여'].min()) # 최소값

print('합계값 = ', df['급여'].sum()) # 합계값

print('평균값 = ', df['급여'].mean()) # 평균값

print('중앙값 = ', df['급여'].median()) # 중앙값

print('최빈값 = ', df['종목'].mode()) # 최빈값

print('데이터 개수 = ', df['종목'].count()) # 데이터 개수

print('값들의 개수 = \n', df['종목'].value_counts()) # 각 데이터 값들의 개수

print('통계량 = \n', df['급여'].describe()) # 통계량

print('다중 집계 = ', df['급여'].agg('mean')) # 다중 집계

print('그룹 통계 = \n', df.groupby('종목').count()) # 그룹별 통계

df.describe() # csv 파일의 다양한 통계량을 간단하게 확인
```

[Pandas 실습] CSV 파일 데이터 정제하기

	A	B	C	D	E	F	G
1	번호	성명	나이	몸무게	급여	종목	생년월일
2	100	김민재	27	75	5428000	축구	1996-11-15
3	101	이강인	22	57	3428	축구	2001-02-09
4	102	박찬호	500	91	8428000	야구	1973-07-28
5	103	차범근	70		4428000	축구	1953-05-22
6	104	추신수		100	4528000	야구	1982-07-03
7	105	손흥민	31	72	7028000	축구	1992-07-08
8	106	황희찬	28	69	2528000	축구	1996-01-29
9	106	황희찬	28	69	2528000	축구	1996-01-29

- 데이터 정제는 데이터 집합에서 잘못된 데이터를 수정하는 것을 의미

- 잘못된 데이터 유형

- 빈 셀 (Empty cells)
- 잘못된 값 (Wrong data) : 극단치 (Outlier)
- 잘못된 형식의 데이터 (Wrong format data)
- 중복 입력된 데이터 (Duplicate data)

```
# 극단치 및 결측치가 존재하는 '나이' 컬럼을 삭제
df.drop(columns=['나이']) # 나이 컬럼 삭제하기
df.drop(1) # 1번 인덱스 삭제하기
df.drop([3, 4]) # 2번과 3번 인덱스 삭제하기
df.drop(df.index[1:4]) # 3~5번 인덱스 삭제하기
```

```
# '몸무게' 컬럼에 비어 있는 모든 셀을 80으로
df['몸무게'] = df['몸무게'].fillna(80)
```

```
# 빈 셀을 대체하는 일반적인 방법은 해당 컬럼의 데이터를 대표하는 평균,
중앙값을 계산
avg = df['몸무게'].mean() # '몸무게' 컬럼의 평균을 구하기
df['몸무게'] = df['몸무게'].fillna(avg) # '몸무게' 컬럼의 빈 셀을 평균값으로
바꾸기
```

```
# '나이' 컬럼에 500 과 같이 극단치가 입력되어 있다면, 이는 분명히 50을 잘 못
입력한 것으로 판단할 수 있음
df.loc[2, '나이'] = 50 # 극단값을 새로운 값으로 변경
```

```
# '급여' 컬럼에는 3428 이라는 너무 작은 값이 포함.
avg = df['급여'].mean() # '급여' 컬럼의 평균값을 구하기
df.loc[1, '급여'] = int(avg) # '급여' 컬럼의 극단치(작은값)를 평균값으로 변경
```

```
# 중복된 행을 찾아 True, False를 반환
df.duplicated()
df.drop_duplicates(inplace=True) # 중복된 행을 삭제
```

[Pandas 실습] CSV 파일 데이터 분석 과제

- 연습문제 데이터 파일 : [titanic_kor.csv](#)
- 타이타닉호의 침몰은 역사상 가장 참혹한 선박 침몰 사고 중 하나이다.
- 1912년 4월 15일, 첫 항해 중 절대 "침몰할 수 없는" 것으로 여겨졌던 RMS 타이타닉호가 빙산과 충돌한 후 침몰하게 되었다. 불행히도 탑승객 모두를 태울 구명보트가 충분하지 않아 승객과 승무원 2,224명 중 1,502명이 사망하게 되었다.
- 살아남는 데는 어느 정도 운이 작용하겠지만, 어떤 집단은 다른 집단보다 생존 가능성이 더 큰 듯 보였다.
- 아래의 승객 데이터(이름, 나이, 성별, 사회경제적 계층 등)를 사용하여 "어떤 사람들이 생존 가능성이 더 높을까?"라는 질문에 답할 수 있는 데이터 분석을 수행해 보자.

- 생존 : 생존여부 (0=사망, 1=생존)
- 등급 : 티켓 클래스(1=1등석, 2=2등석, 3=3등)
- 성별 : 성별
- 나이 : 나이
- 형제자매 : 함께 탑승한 형제/자매
- 부모자녀 : 함께 탑승한 부모/자녀
- 요금 : 탑승 요금
- 탑승지코드 : 탑승지 코드
- 객실등급 : 객실 등급(1=1등석, 2=2등석, 3=3등석)
- 성인여부 : 남여 성인 여부
- 성인남성 : 성인 남성 여부
- 선실코드 : 선실 코드
- 탑승지명 : 선착장
- 생존여부 : 생존 여부(no=사망, yes=생존)
- 개인탑승자 : 개인 탑승자 여부

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	생존	등급	성별	나이	형제자매	부모자녀	요금	탑승지코드	객실등급	성인여부	성인남성	선실코드	탑승지명	생존여부	개인탑승자
2	0	3	male	22	1	0	7.25	S	Third	man	TRUE		Southampton	no	FALSE
3	1	1	female	38	1	0	71.2833	C	First	woman	FALSE	C	Cherbourg	yes	FALSE
4	1	3	female	26	0	0	7.925	S	Third	woman	FALSE		Southampton	yes	TRUE
5	1	1	female	35	1	0	53.1	S	First	woman	FALSE	C	Southampton	yes	FALSE
6	0	3	male	35	0	0	8.05	S	Third	man	TRUE		Southampton	no	TRUE
7	0	3	male		0	0	8.4583	Q	Third	man	TRUE		Queenstown	no	TRUE
8	0	1	male	54	0	0	51.8625	S	First	man	TRUE	E	Southampton	no	TRUE
9	0	3	male	2	3	1	21.075	S	Third	child	FALSE		Southampton	no	FALSE
10	1	3	female	27	0	2	11.1333	S	Third	woman	FALSE		Southampton	yes	FALSE
11	1	2	female	14	1	0	30.0708	C	Second	child	FALSE		Cherbourg	yes	FALSE
12	1	3	female	4	1	1	16.7	S	Third	child	FALSE	G	Southampton	yes	FALSE

[Pandas 실습] CSV 파일 데이터 분석 과제

- 1) `pands`, `numpy`, `csv`, `matplotlib` 라이브러리를 불러오시오.
- 2) 타이타닉 (`titani`) 데이터셋을 불러오시오.
- 3) 데이터셋에서 처음 3개 행을 출력하시오.
- 4) 데이터셋의 컬럼 이름을 출력하시오.
- 5) 데이터셋에 대한 통계자료를 출력하시오.
- 6) 데이터셋의 각 컬럼에 대한 데이터 형식(`int`, `object`, `float...`)을 출력하시오.
- 7) '객실등급' 컬럼과 '요금' 컬럼에 대한 데이터만 출력하시오.
- 8) 인덱스 10 ~ 15번 행에 대한 데이터만 출력하시오.
- 9) '객실등급' 컬럼과 '요금' 컬럼에 대한 인덱스 21 ~ 23번 행의 데이터만 출력하시오.

[Pandas 실습] CSV 파일 데이터 분석 과제

- 10) 객실 요금이 300 불 이상인 데이터에 대한 '성별', '나이', '객실등급', '생존여부' 컬럼을 출력하시오.
- 11) '객실등급'의 데이터 값을 모두 소문자로 바꾸고 출력하시오. (First -> first, Second -> second, Third -> third)
- 12) '성인여부' 컬럼의 데이터 값 중 man, woman 을 adult 로 바꾸시오.
- 13) 데이터 셋의 각 컬럼에 대한 공백(결측치)의 갯수를 출력하시오.
- 14) 데이터셋에는 총 몇 건의 데이터가 저장되어 있는지 출력하시오 (전체 탑승자는 몇 명인가).
- 15) '생존여부' 컬럼의 데이터와 비교하여 '생존' 컬럼의 데이터 값이 동일한지를 확인하시오. ('생존' 컬럼의 데이터가 1 일때, '생존여부' 컬럼의 데이터가 no 이라면 서로 다른 데이터이다. 해당 데이터가 있는지를 찾으면 된다.)
- 16) '생존' 컬럼과 동일한 값이 저장되어 있는 '생존여부' 컬럼을 삭제하시오. 그리고 '등급' 컬럼과 동일한 값이 저장되어 있는 '객실등급' 컬럼을 삭제하시오.

[Pandas 실습] CSV 파일 데이터 분석 과제

- 17) '탑승지코드' 컬럼과 '탑승지명' 컬럼 모두에서 데이터 값이 누락되어 있는 행(인덱스)을 찾으시오.
- 18) '탑승지코드' 컬럼과 '탑승지명' 컬럼에서 결측치(데이터가 누락되어 있는)가 존재하는 행을 삭제하시오. (인덱스 61, 829 번 행에 결측치가 존재한다.)
- 19) 인덱스 59 ~ 63번 행에 대한 데이터만 출력하시오.
- 20) '성인여부' 컬럼에서 **child** 데이터에 대응하는 '나이' 컬럼의 데이터가 누락되어 있는지 확인하시오. ('성인여부' 컬럼의 데이터가 **child** 일때, '나이' 컬럼의 데이터가 **isnull** 인 데이터를 찾으면 된다.)
- 21) '나이' 컬럼의 데이터 값에 대한 평균을 출력하시오.
- 22) '나이' 컬럼의 결측치(빈 셀)를 '나이'에 대한 평균 값으로 채우시오.
- 23) '선실코드' 컬럼의 데이터 값에는 어떤 종류의 값이 있는지 출력하시오 (7 개).

[Pandas 실습] CSV 파일 데이터 분석 과제

24) '요금' 컬럼에서 가장 높은 요금이 얼마인지 출력하시오.

25) 생존자에 대한 데이터만 출력하시오. ('생존' 컬럼의 데이터 값이 1 인 데이터를 추출하여 'survivor' 변수에 저장한 다음 출력하시오.)

26) '객실등급' 별 승객수를 출력하시오.

27) '객실등급' 별 최고 요금을 출력하시오.

28) '객실등급'별 승객의 평균 나이를 출력하시오.

29) '생존'과 '나이' 컬럼을 기준으로 내림차순 정렬한 다음, 20개의 데이터만 표시하시오.

30) 여성 생존율과 남성 생존율을 아래와 같이 표시하시오.

여성 생존율 = 0.74

남성 생존율 = 0.19

[Pandas 실습] CSV 파일 데이터 분석 과제

목표 : 승객 데이터(이름, 나이, 성별, 사회경제적 계층 등)를 사용하여 "어떤 사람들이 생존 가능성이 더 높을까?" 라는 질문에 답할 수 있는 데이터 분석을 수행

1단계. 문제 정의

- 목표: 타이타닉 탑승객 중 어떤 특성을 가진 사람들이 생존할 확률이 높았는지 파악
- 주요 변수 후보: 성별, 나이, 객실등급, 탑승지, 요금, 동승자 수 등

2단계. 데이터 이해 및 전처리

- 결측치 처리: 나이, 탑승지코드, 선실코드 등에 결측치가 있는지 확인하고 평균/중앙값 대체 또는 삭제
- 데이터 변환: 범주형 → 숫자형 변환 (예: 성별 `male=0`, `female=1`). 필요 없는 중복된 컬럼 제거 (이전 과제에서 한 것처럼)
- 파생 변수 생성:
 $\text{가족수} = \text{형제자매수} + \text{부모자녀수} + 1$
 $\text{연령대} = (\text{어린이/청년/중년/노인 등})$

[Pandas 실습] CSV 파일 데이터 분석 과제

3단계. 탐색적 데이터 분석 (EDA)

- 기초 통계: 생존/사망자 수 비율 확인
- 시각화:
 - 성별 vs 생존율 (막대그래프)
 - 객실등급 vs 생존율 (막대그래프)
 - 나이 분포와 생존 여부 (히스토그램, KDE Plot)
 - 요금 vs 생존 여부 (박스플롯)
- 교차분석:
 - 성별 + 객실등급 → 생존율
 - 가족 유무 → 생존율

4단계. 통계적 분석

- 성별/객실등급과 생존 여부의 관계 유의성 확인
- 생존자 vs 사망자의 나이 평균 비교

[Pandas 실습] CSV 파일 데이터 분석 과제

5단계. 해석 및 결론

- “여성, 1등석 승객, 어린이, 요금이 높은 승객, 가족이 함께 있는 경우” → 생존 확률 70%
- “남성, 3등석 승객, 나이가 많은 승객” → 생존 확률 20%

[확장. AI 예측 모델링]

- 로지스틱 회귀: 생존 여부에 영향을 주는 요인을 계수로 확인
- 의사결정나무/랜덤포레스트: 변수 중요도 확인
- 교차검증: 모델 성능 평가